

Supervisión y Alertas de Instancias EC2 con Amazon CloudWatch

 **Dificultad:** Intermedio / Operaciones y Seguridad

 **Tiempo Estimado:** 60 minutos

 **Servicios Principales:** Amazon CloudWatch, Amazon Simple Notification Service (SNS), Amazon EC2.

Resumen y Objetivos

En este laboratorio, implementarás una arquitectura de monitoreo reactivo para detectar comportamientos anómalos en tus servidores. Configurarás notificaciones automáticas mediante **Amazon SNS** y crearás alarmas en **Amazon CloudWatch** basadas en umbrales de uso de CPU. Finalmente, simularás un ataque de secuestro de recursos (como *cryptojacking* o *malware*) estresando la instancia al 100% para validar que el sistema de alertas funcione correctamente.

Objetivos alcanzados al finalizar:

-  Crear un tema y una suscripción de correo electrónico en Amazon SNS.
 -  Configurar una alarma en CloudWatch basada en la métrica **CPUUtilization**.
 -  Ejecutar una prueba de estrés (*Stress Test*) en una instancia EC2 a través de una terminal segura.
 -  Validar la recepción de la alerta de seguridad por correo electrónico.
 -  Construir un Panel de Control (*Dashboard*) en CloudWatch para el monitoreo visual en tiempo real.
-

Análisis del Escenario (Diagnóstico Inicial)

El registro (Logging) y el monitoreo (Monitoring) son vitales para identificar "banderas rojas" de seguridad que a menudo pasan desapercibidas.

Diagnóstico y Estrategia: Un pico repentino y sostenido de CPU en un servidor web que normalmente tiene baja carga no solo es un problema de rendimiento, a menudo es un indicador de compromiso (IoC). Un actor malintencionado podría haber inyectado *malware* para minar criptomonedas o ejecutar ataques de denegación de servicio (DDoS) desde tu infraestructura. Como Ingeniero Cloud, tu tarea es automatizar la visibilidad de este servidor. Crearás una alarma que, al superar el 60% de uso de CPU de forma anómala, dispare una alerta inmediata a los administradores.

Arquitectura del Laboratorio

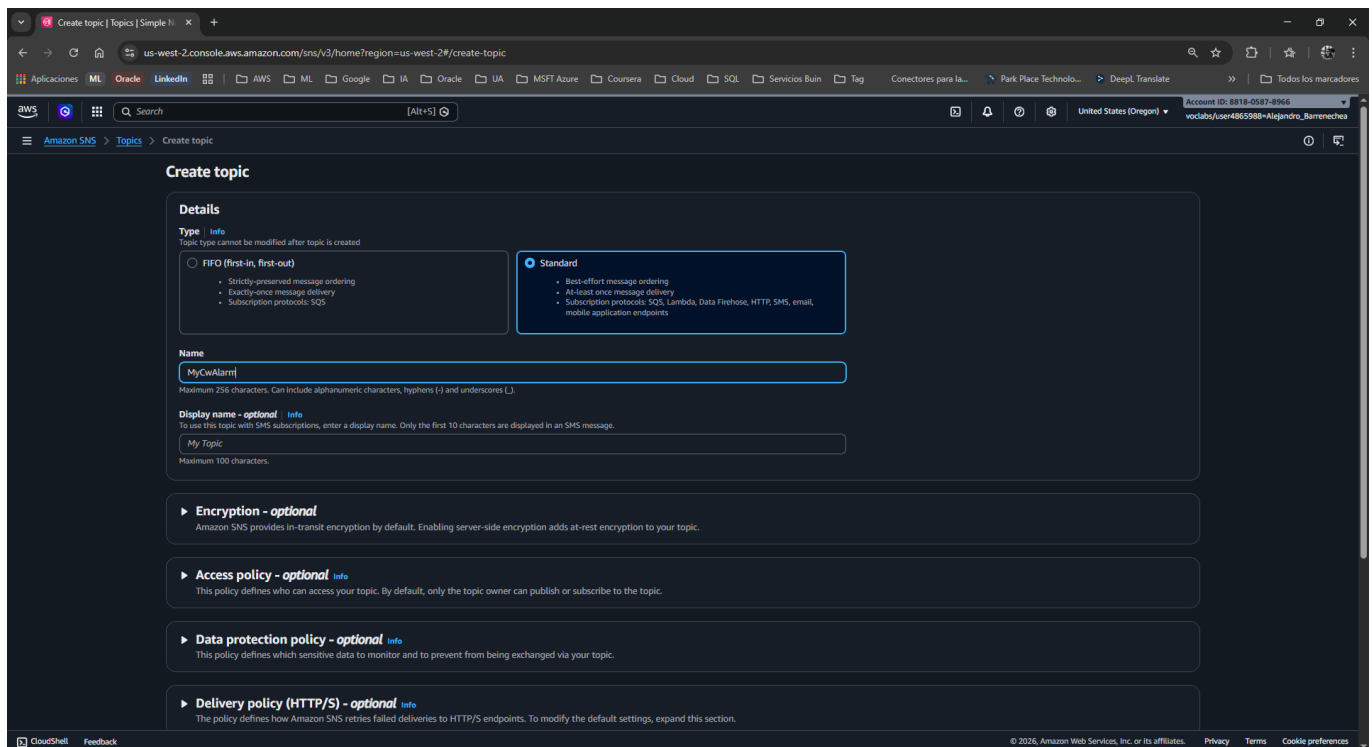
1.  **Amazon EC2 ("Stress Test"):** Servidor Linux objetivo que emitirá telemetría constante (métricas).
 2.  **Amazon CloudWatch:** El "cerebro" del monitoreo que recopila la métrica **CPUUtilization**. Contiene la Alarma que evalúa el umbral y el *Dashboard* visual.
 3.  **Amazon SNS:** Servicio de mensajería (Pub/Sub) que recibe el cambio de estado de la alarma de CloudWatch y "empuja" (Push) la notificación al correo del administrador.
-

⚙️ Desarrollo de las Tareas (Paso a Paso)

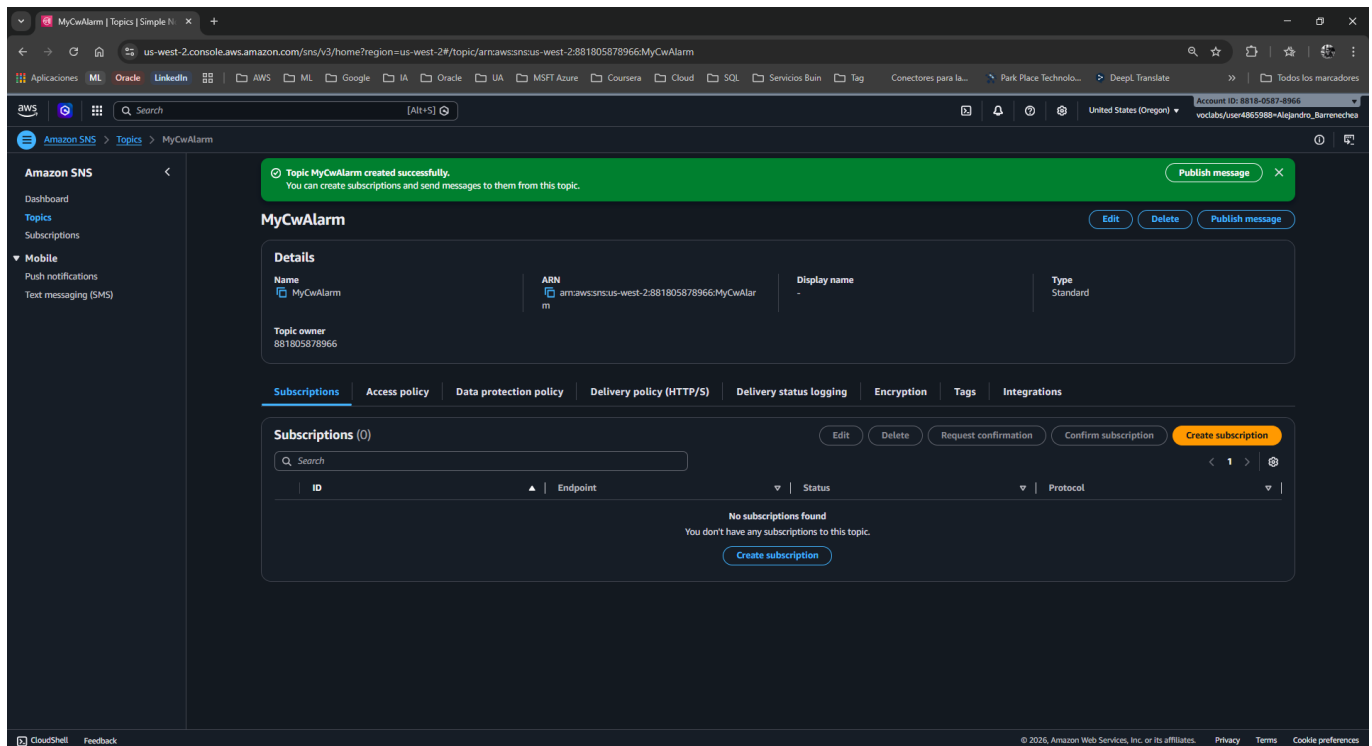
📧 Tarea 1: Configurar Amazon SNS (Notificaciones)

Primero, crearás el canal de comunicación que entregará las alertas.

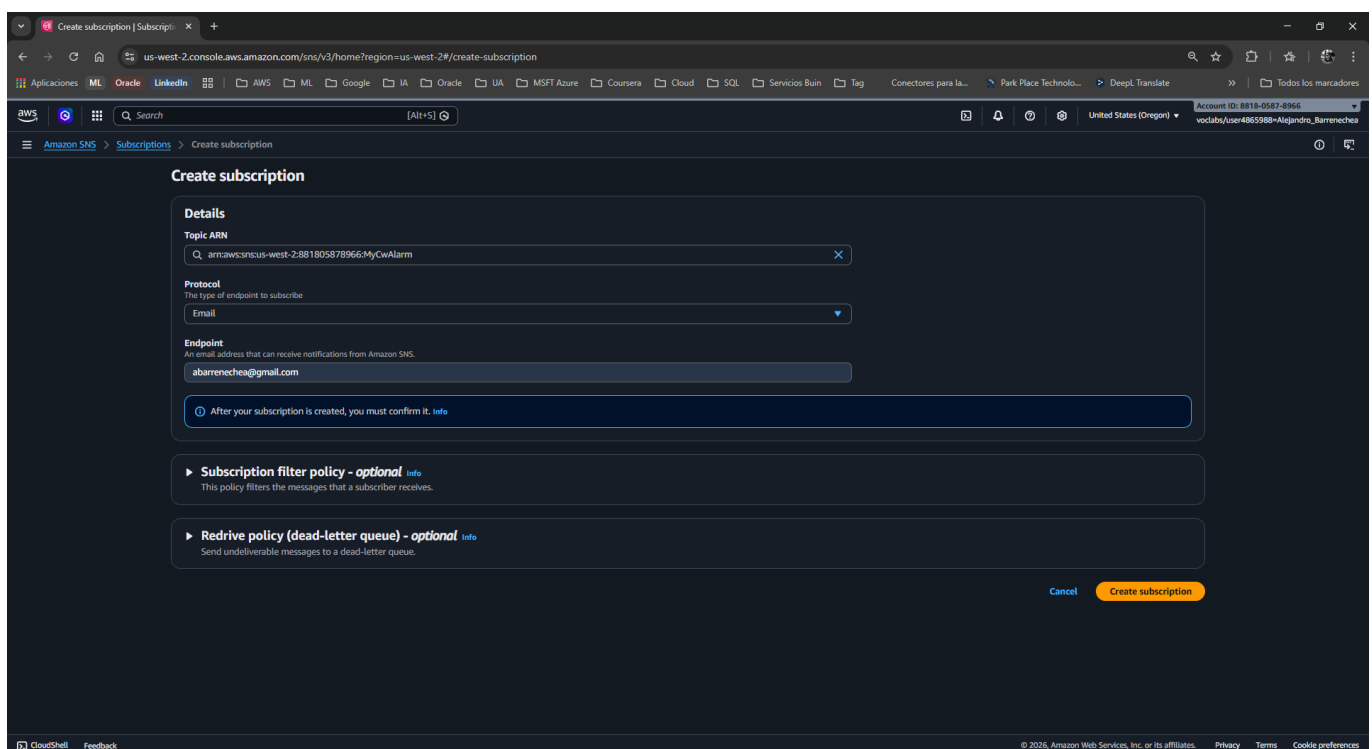
1. 🔍 En la barra de búsqueda de la consola, escribe **SNS** y selecciona **Simple Notification Service**.
2. 📁 En el panel izquierdo, haz clic en el ícono de las tres líneas (menú), selecciona **Topics** (Temas) y haz clic en **Create topic** (Crear tema).
3. ⚙️ En la sección *Details*, configura lo siguiente:
 - **Type:** Selecciona **Standard** (Estándar).
 - **Name:** Escribe **MyCwAlarm**
4. 📄 Desplázate hacia abajo y haz clic en **Create topic**.



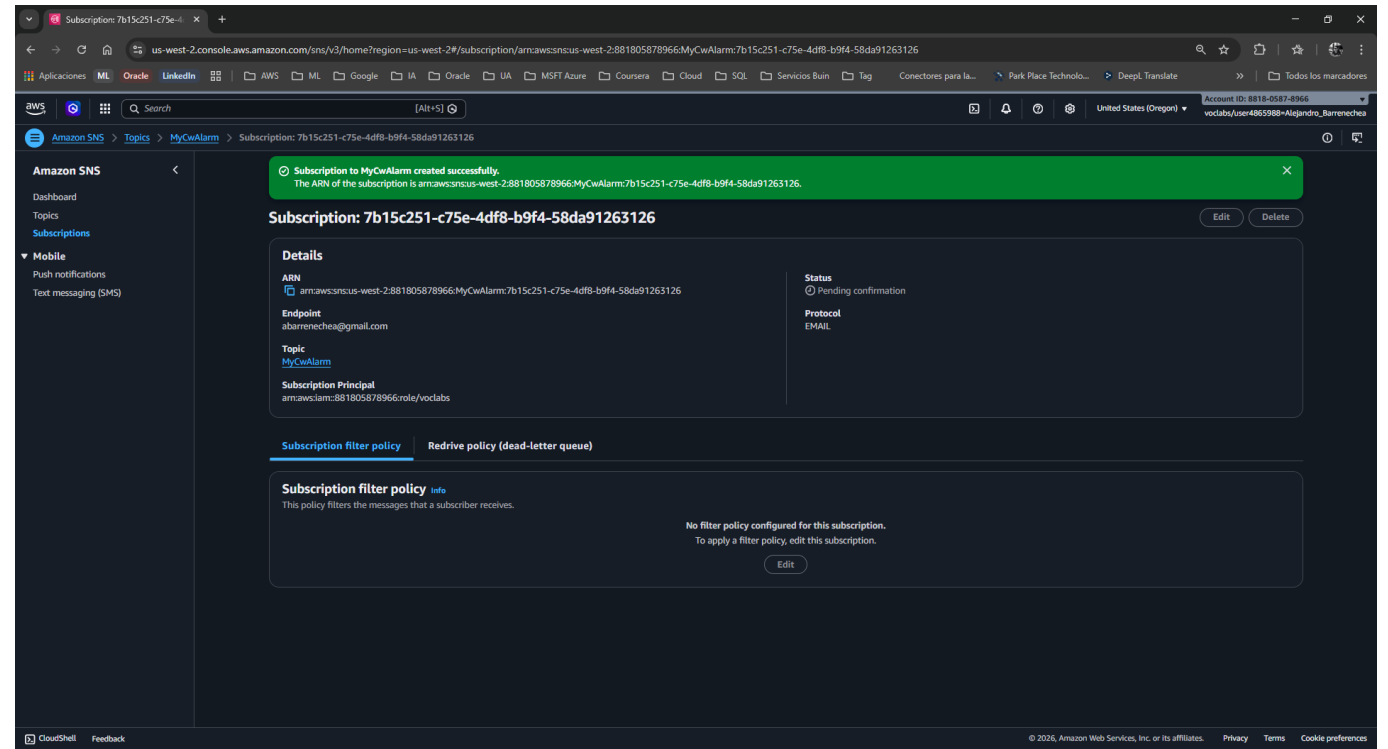
The screenshot shows the Amazon SNS 'Create topic' console page. The page is titled 'Create topic' and has a breadcrumb trail 'Amazon SNS > Topics > Create topic'. The 'Details' section is active, showing the 'Type' as 'Standard' (selected) and 'Name' as 'MyCwAlarm'. The 'Name' field has a maximum length of 256 characters. Below the 'Name' field, there is a 'Display name - optional' field with the value 'My Topic' and a maximum length of 100 characters. The 'Encryption - optional' section is expanded, showing that Amazon SNS provides in-transit encryption by default. The 'Access policy - optional' section is also expanded, showing that the policy defines who can access the topic. The 'Data protection policy - optional' section is expanded, showing that the policy defines which sensitive data to monitor and to prevent from being exchanged via the topic. The 'Delivery policy (HTTP/S) - optional' section is expanded, showing that the policy defines how Amazon SNS retries failed deliveries to HTTP/S endpoints. The page footer includes 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates.



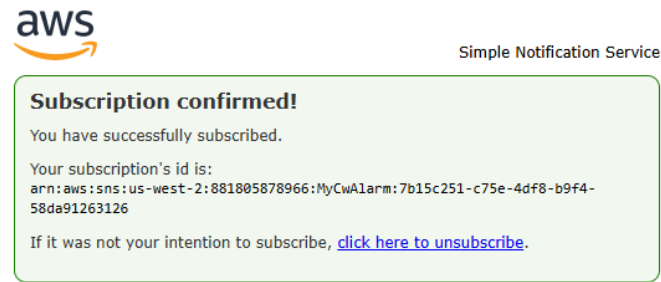
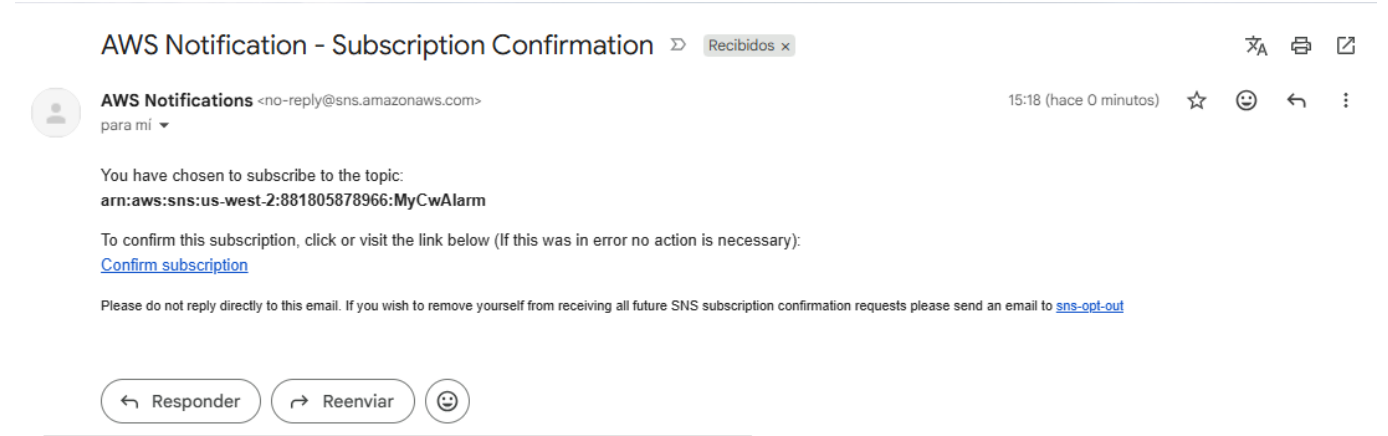
5. 탭 En la página de detalles de tu nuevo tema **MyCwAlarm**, selecciona la pestaña **Subscriptions** (Suscripciones) y haz clic en **Create subscription**.
6. ⚙ En la nueva pantalla, configura la suscripción:
 - **Topic ARN:** Déjalo por defecto (ya está seleccionado tu tema).
 - **Protocol:** En el menú desplegable, selecciona **Email** (Correo electrónico).
 - **Endpoint:** Escribe una dirección de correo electrónico real a la que tengas acceso en este momento.
7. + Haz clic en **Create subscription**.



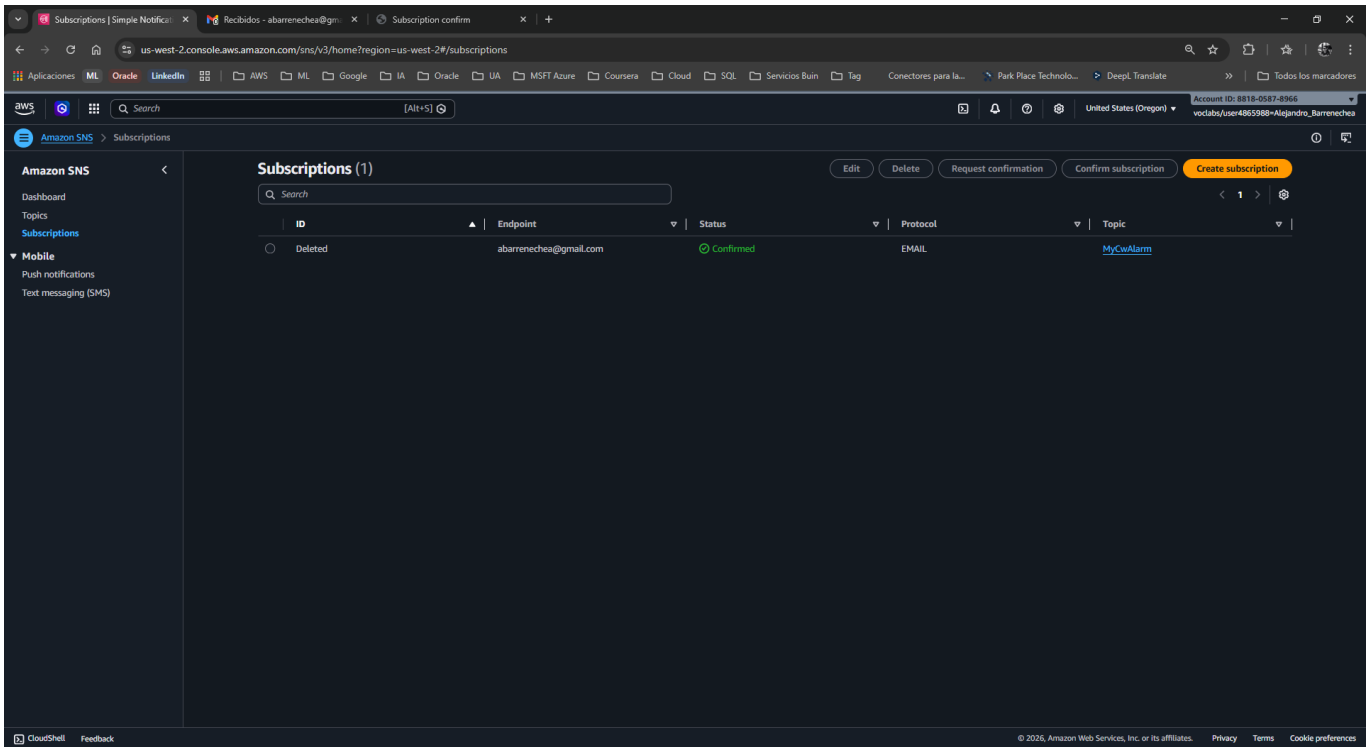
8. 👁 Notarás que el estado (*Status*) dice *Pending confirmation* (Pendiente de confirmación).



9. ✉ Abre tu correo electrónico en otra pestaña, busca el mensaje titulado **AWS Notification - Subscription Confirmation** y haz clic en el enlace **Confirm subscription**.



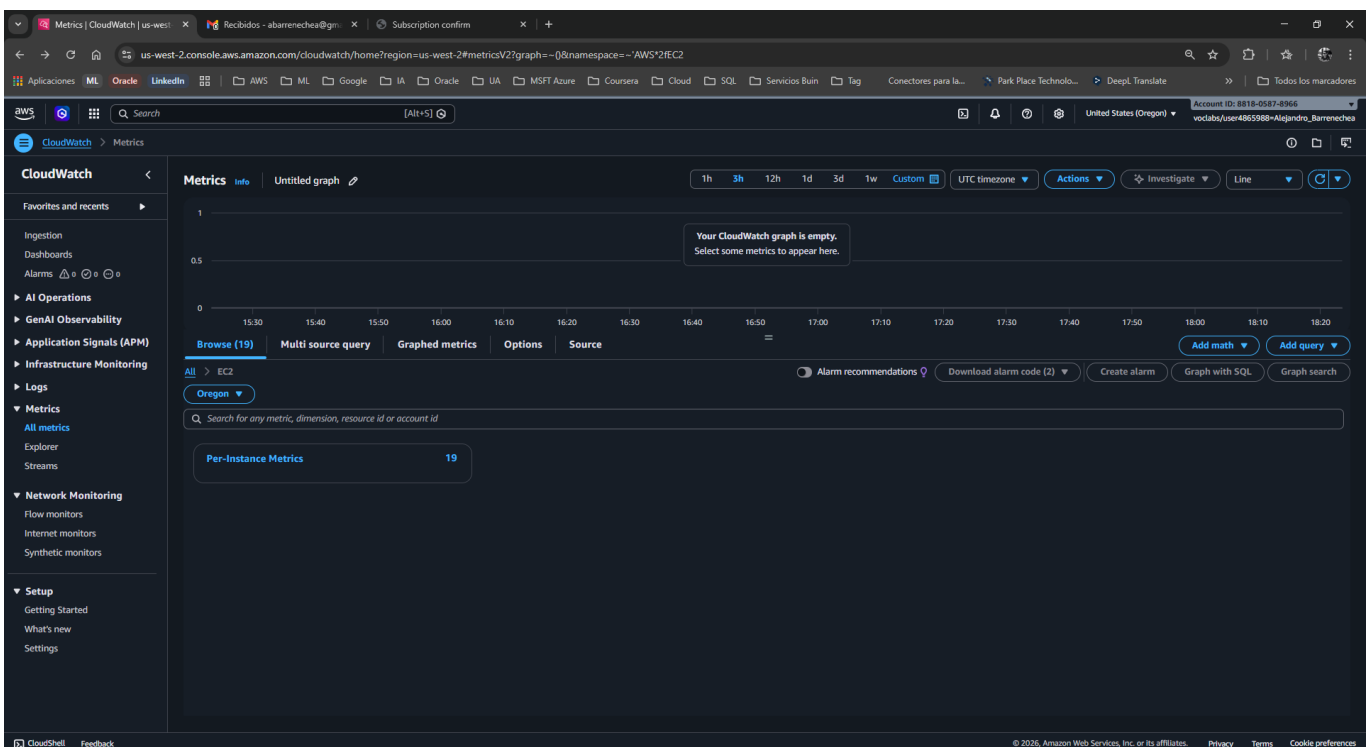
10. 🏠 Vuelve a la consola de AWS, ve al panel izquierdo, selecciona **Subscriptions** y verifica que el estado ahora sea verde y diga **Confirmed**.



Tarea 2: Crear una alarma de CloudWatch

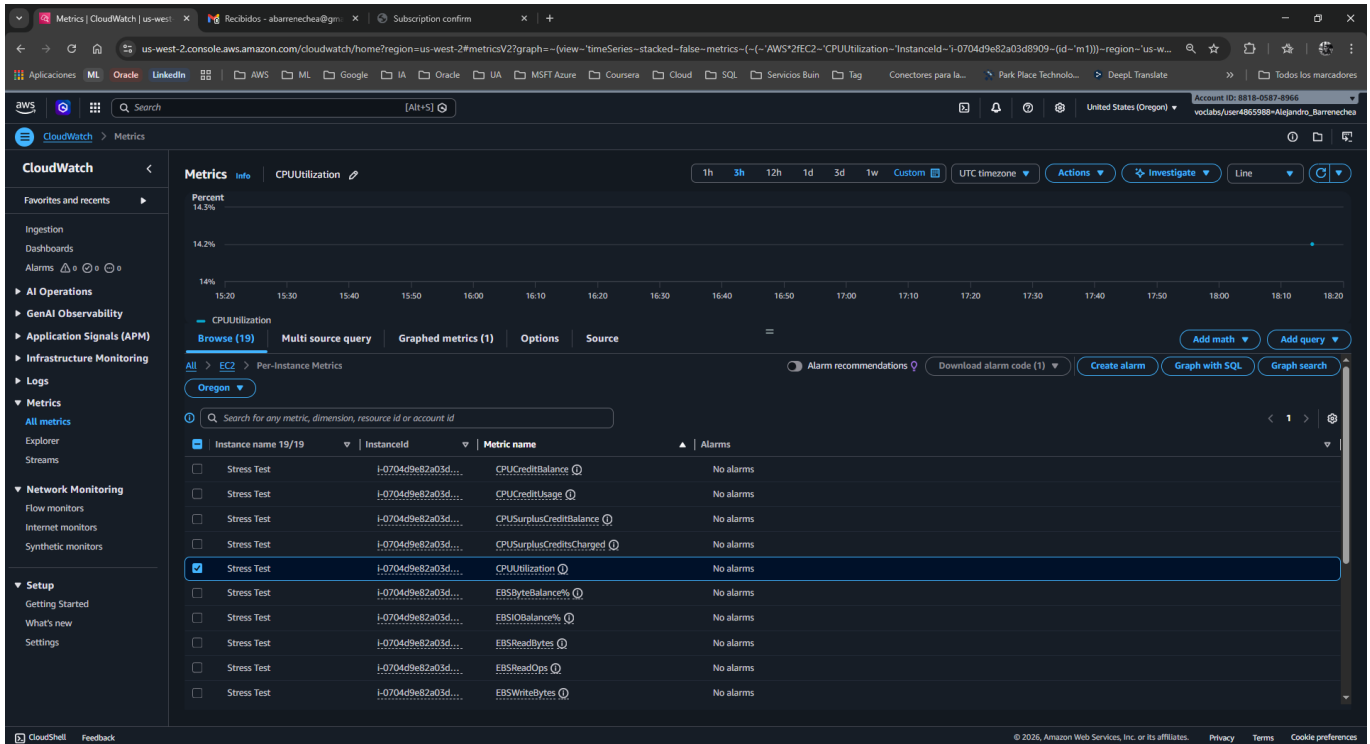
Ahora le dirás a AWS qué métrica observar y cuándo disparar la notificación.

1. 🔍 En la barra de búsqueda superior, escribe **CloudWatch** y abre el servicio.
2. 📁 En el panel izquierdo, despliega la sección **Metrics** (Métricas) y selecciona **All metrics** (Todas las métricas).
 - *Nota:* CloudWatch puede tardar unos 5 minutos en empezar a mostrar las métricas de una instancia recién creada.

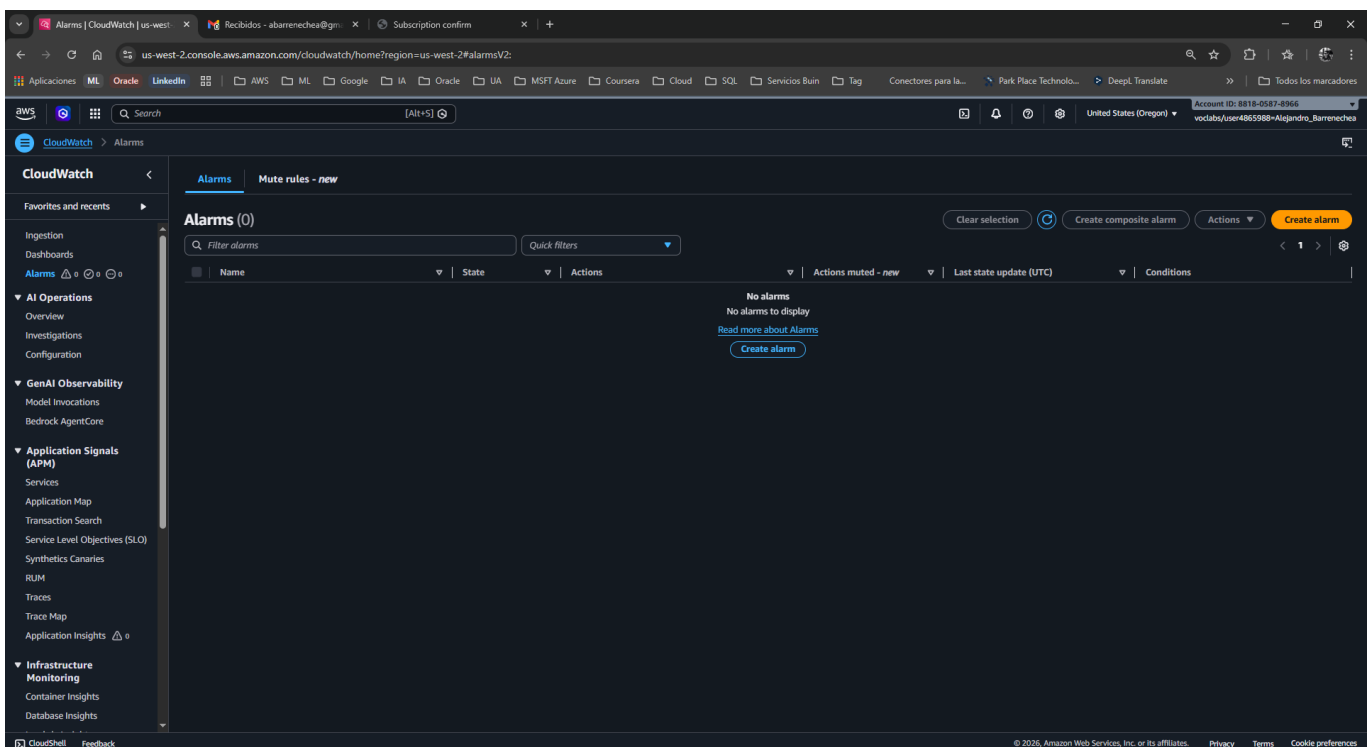


3. 🖱️ En la tarjeta de exploración, haz clic en **EC2** y luego en **Per-Instance Metrics** (Métricas por instancia).

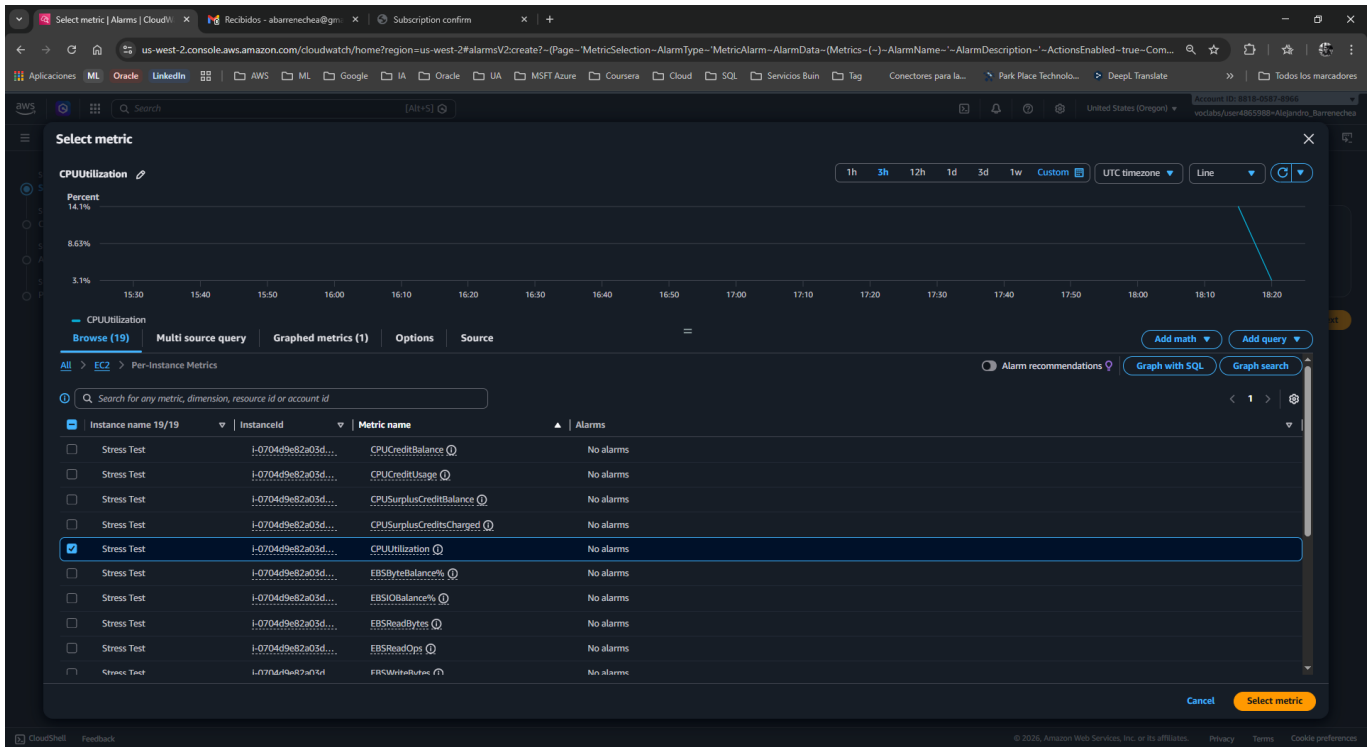
4. ☒ Utiliza el cuadro de búsqueda para encontrar la instancia llamada **Stress Test** y marca la casilla correspondiente a la métrica **CPUUtilization** (Uso de CPU). Verás un gráfico plano cerca del 0%.



5. ☐ En el panel izquierdo, despliega la sección **Alarms** (Alarmas) y selecciona **All alarms**.



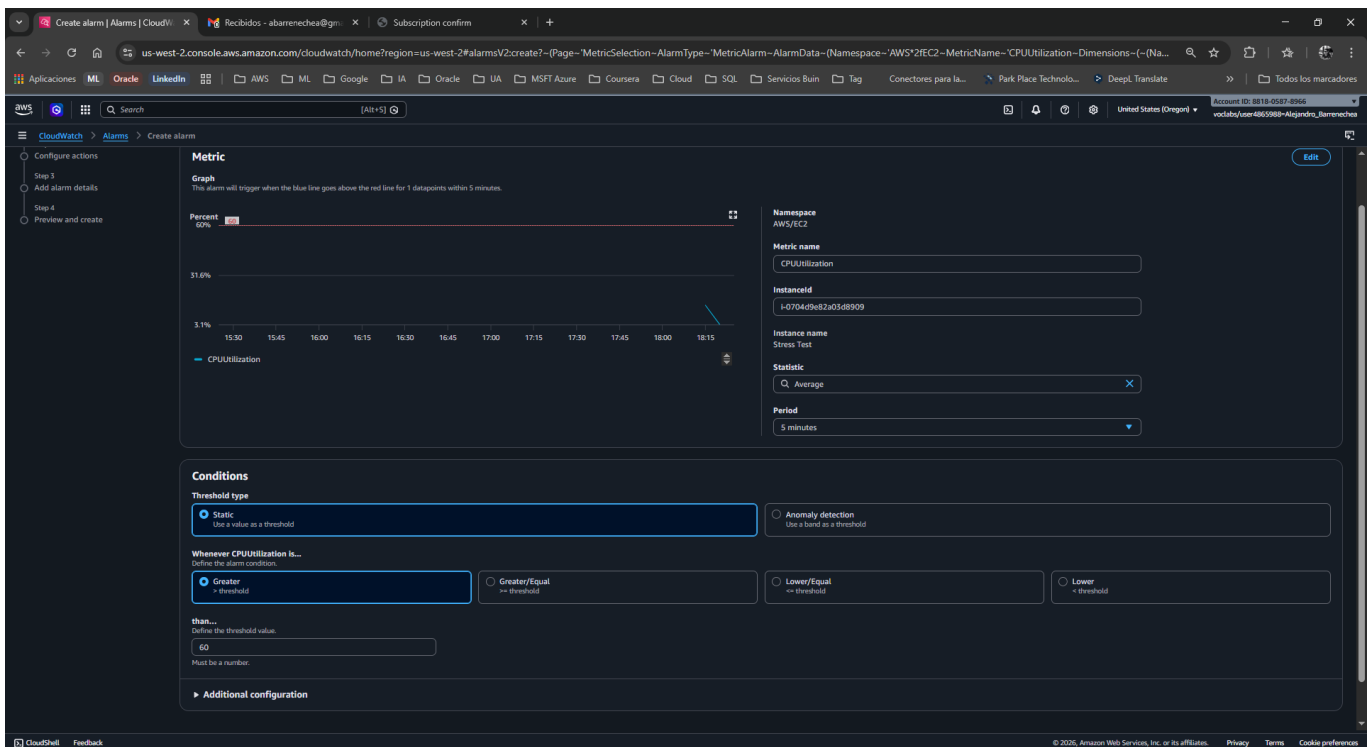
6. Haz clic en **Create alarm** (Crear alarma).
7. Haz clic en **Select metric > EC2 > Per-Instance Metrics**, marca la casilla de **CPUUtilization** para la instancia "Stress Test" y haz clic nuevamente en **Select metric**.



8. En la pantalla *Specify metric and conditions*, baja a la sección **Conditions** y configura el gatillo:

- **Threshold type:** **Static**
- **Whenever CPUUtilization is...:** Selecciona **Greater > threshold** (Mayor que).
- **than...:** Escribe **60** (representa el 60%).

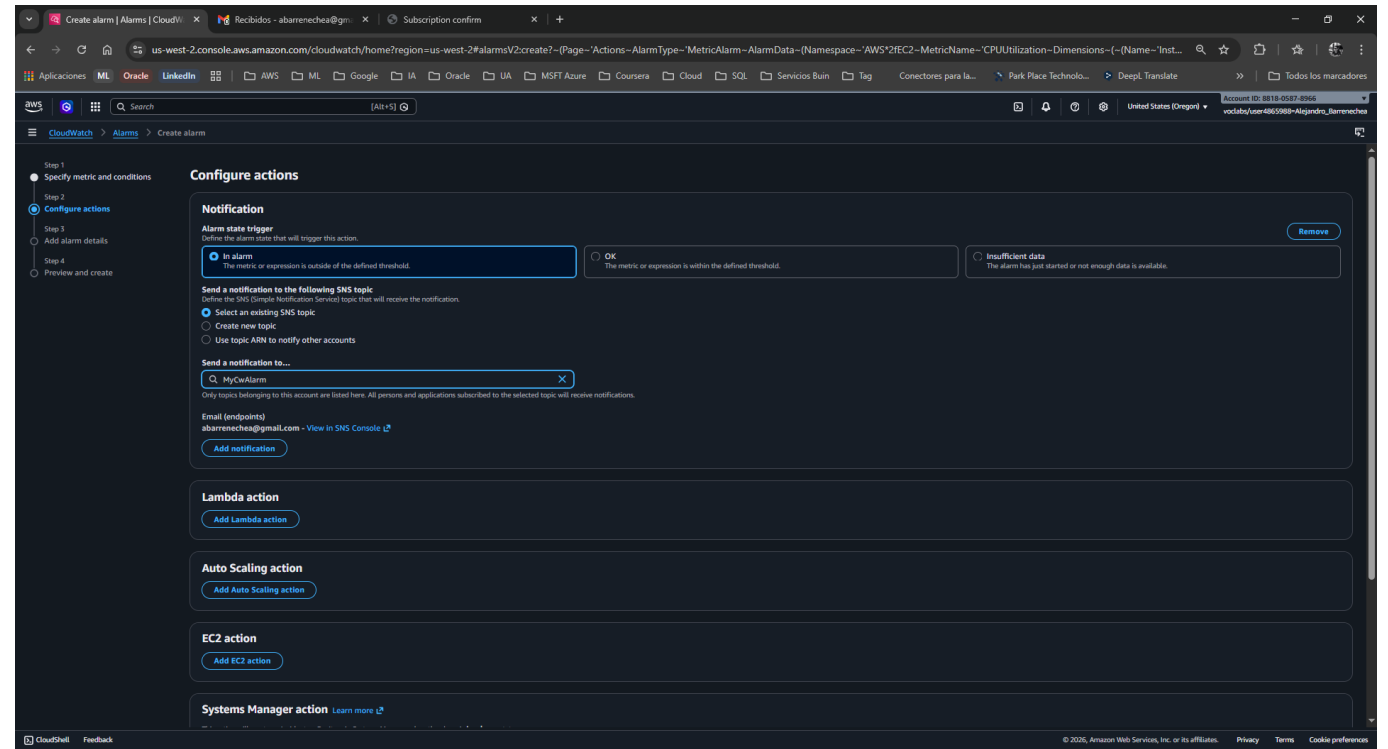
9. Haz clic en **Next**.



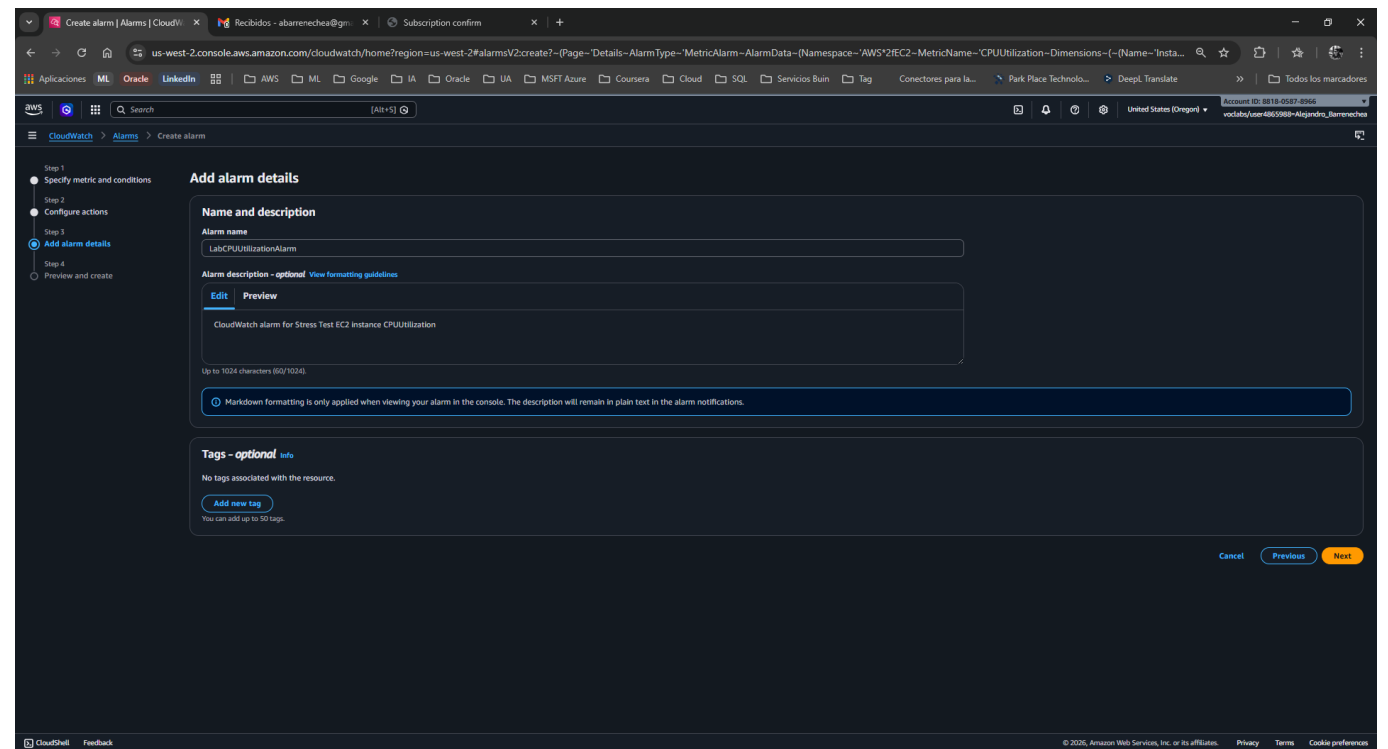
10. En la pantalla *Configure actions*, configura la notificación:

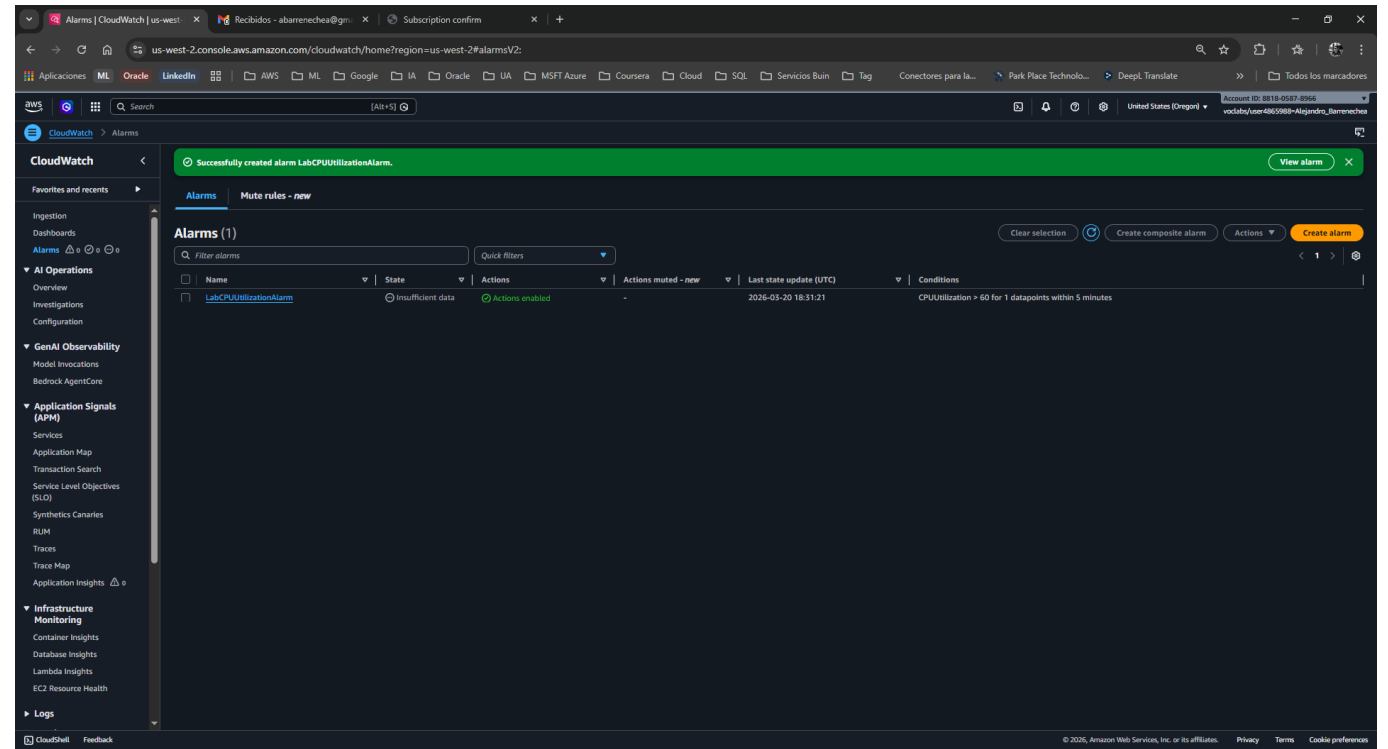
- **Alarm state trigger:** **In alarm** (En alarma).
- **Select an SNS topic:** Selecciona **Select an existing SNS topic**.
- **Send a notification to...:** Haz clic en el cuadro y elige tu tema **MyCwAlarm**.

11. Haz clic en **Next**.



12. ✎ Asigna el nombre y descripción:
- **Alarm name:** LabCPUUtilizationAlarm
 - **Alarm description:** CloudWatch alarm for Stress Test EC2 instance CPUUtilization
13. ➡ Haz clic en **Next**, revisa la configuración y haz clic en **Create alarm**.





 Tarea 3: Probar la Alarma (Simulación de Malware)

Simularás un ataque inyectando un comando que sature el procesador para validar que tu defensa reaccione.

- 1.  Vuelve a la pestaña de tu plataforma (Vocareum) y haz clic en el botón **AWS Details**.
- 2.  Junto a **EC2InstanceURL**, copia el enlace provisto y pégalo en una nueva pestaña del navegador. Esto te conectará directamente a la terminal del servidor por *Session Manager*.

Cloud Access

AWS CLI: Show

Cloud Labs

Remaining session time: 00:39:53(40 minutes)

Session started at: 2026-03-20T11:11:56-0700

Session to end at: 2026-03-20T12:11:56-0700

Accumulated lab time: 00:20:00 (20 minutes)

ips -- public:54.201.238.181, private:10.0.2.177

SSH key Show Download PEM Download PPK

AWS SSO Download URL

EC2InstanceURL	https://us-west-2.console.aws.amazon.com/systems-manager/session-manager/i-0704d9e82a03d8909?region=us-west-2
Region	us-west-2

- 3.  Para aumentar artificialmente la carga de la CPU, ejecuta el siguiente comando de prueba de estrés:

```
sudo stress --cpu 10 -v --timeout 400s
```

- *Diagnóstico:* Este comando generará 10 subprocesos (workers) saturando la CPU al 100% durante 400 segundos (casi 7 minutos).

```
sh-4.2$ sudo stress --cpu 10 -v --timeout 400s
stress: info: [3431] dispatching hogs: 10 cpu, 0 io, 0 vm, 0 hdd
stress: debug: [3431] using backoff sleep of 30000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 10 [3432] forked
stress: debug: [3431] using backoff sleep of 27000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 9 [3433] forked
stress: debug: [3431] using backoff sleep of 24000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 8 [3434] forked
stress: debug: [3431] using backoff sleep of 21000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 7 [3435] forked
stress: debug: [3431] using backoff sleep of 18000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 6 [3436] forked
stress: debug: [3431] using backoff sleep of 15000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 5 [3437] forked
stress: debug: [3431] using backoff sleep of 12000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 4 [3438] forked
stress: debug: [3431] using backoff sleep of 9000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 3 [3439] forked
stress: debug: [3431] using backoff sleep of 6000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 2 [3440] forked
stress: debug: [3431] using backoff sleep of 3000us
stress: debug: [3431] setting timeout to 400s
stress: debug: [3431] --> hogcpu worker 1 [3441] forked
```

4. Abre una **segunda pestaña** en tu navegador pegando de nuevo el enlace de **EC2InstanceURL**.
5. En esta segunda terminal, monitorea el procesador en vivo ejecutando:

```
top
```

- Verás los procesos "stress" consumiendo cerca del 100% del %CPU.

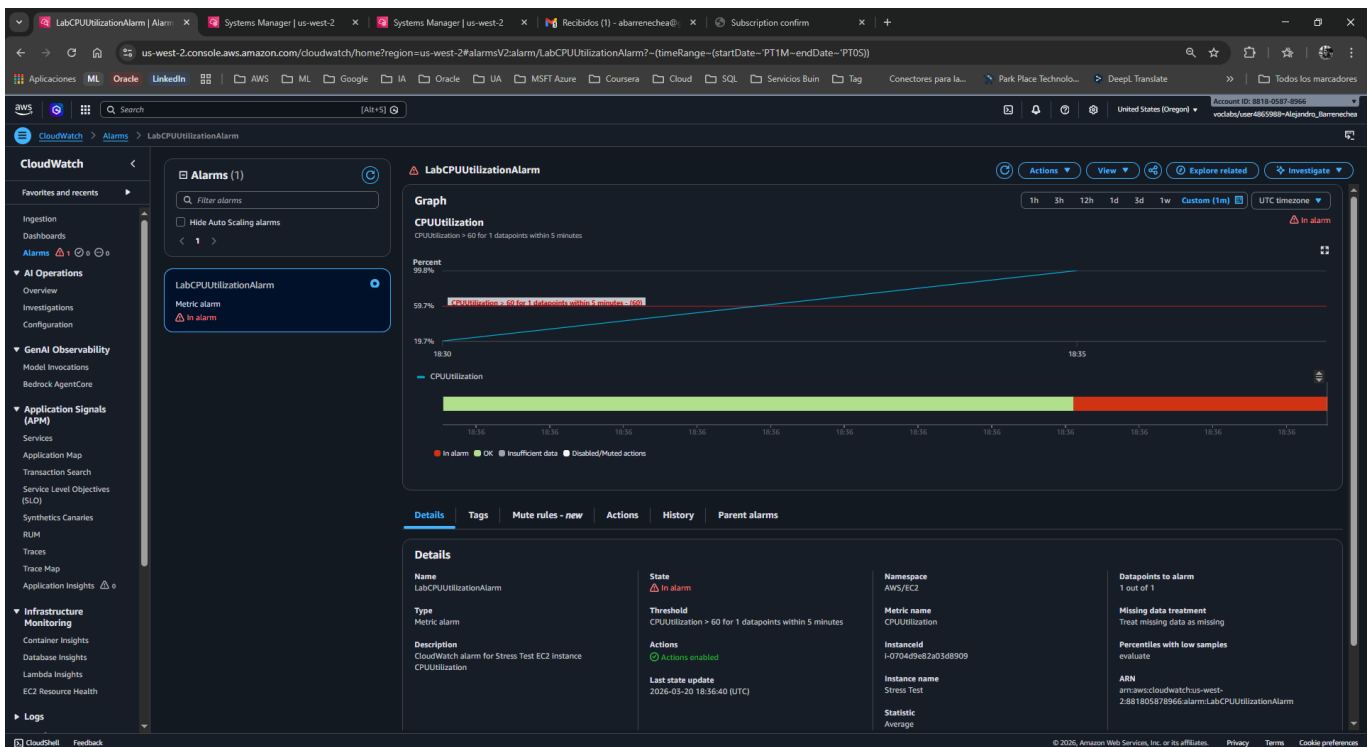
```

top - 18:35:08 up 20 min, 0 users, load average: 0.41, 3.09, 1.12
tasks: 99 total, 11 running, 51 sleeping, 0 stopped, 0 zombie
MiB Mem : 989396 total, 484292 free, 102740 used, 402364 buff/cache
MiB Swap: 0 total, 0 free, 0 used. 745072 avail Mem

PID USER PR NI VIRT RES SHR S CPU MEM TIME COMMAND
3435 root 20 0 7580 96 R 10.0 0.0 0:10.92 stress
3433 root 20 0 7580 96 R 10.0 0.0 0:10.91 stress
3434 root 20 0 7580 96 R 10.0 0.0 0:10.92 stress
3436 root 20 0 7580 96 R 10.0 0.0 0:10.91 stress
3437 root 20 0 7580 96 R 10.0 0.0 0:10.92 stress
3438 root 20 0 7580 96 R 10.0 0.0 0:10.91 stress
3439 root 20 0 7580 96 R 10.0 0.0 0:10.92 stress
3441 root 20 0 7580 96 R 10.0 0.0 0:10.92 stress
3432 root 20 0 7580 96 R 9.7 0.0 0:10.91 stress
3440 root 20 0 7580 96 R 9.7 0.0 0:10.91 stress
3409 root 20 0 1245872 23188 13154 S 0.3 2.4 0:00.11 sm-session-wor
3444 root 20 0 1245872 25432 13168 S 0.3 2.6 0:00.12 sm-session-wor
3457 sm-user 20 0 170948 4560 3972 R 0.3 0.5 0:00.04 top
1 root 20 0 123588 5312 3822 S 0.0 0.6 0:00.74 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kthreadd
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 mm_percpu_wq
7 root 20 0 0 0 0 S 0.0 0.0 0:00.28 ksoftirqd/0
8 root 20 0 0 0 0 I 0.0 0.0 0:00.05 rcu_sched
9 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_bh
10 root rt 0 0 0 S 0.0 0.0 0:00.00 migration/0
11 root rt 0 0 0 S 0.0 0.0 0:00.00 watchdog/0
12 root 20 0 0 0 0 S 0.0 0.0 0:00.00 csdhp/0
14 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kdevtmpfs
15 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 netns
192 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kswapd0
193 root 20 0 0 0 0 S 0.0 0.0 0:00.00 oom_reaper
194 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 writeback
196 root 20 0 0 0 0 S 0.0 0.0 0:00.00 ksmcactid0
197 root 25 5 0 0 0 S 0.0 0.0 0:00.00 ksmd
198 root 39 19 0 0 0 S 0.0 0.0 0:00.00 khugepaged
199 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 cryptd
200 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kintegrityd
202 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kblockd
245 root 20 0 0 0 0 S 0.0 0.0 0:00.00 xen-balloon
356 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 md
559 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 edac-poller
564 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 watchdog
657 root 20 0 0 0 0 I 0.0 0.0 0:00.06 kworker/u30:2
683 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kauditd
707 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kthnifid
802 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 xfsailloc
803 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 xfs_ami_cache
858 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kthnifid
869 root 20 0 0 0 0 S 0.0 0.0 0:00.00 xenbus
870 root 20 0 0 0 0 S 0.0 0.0 0:00.00 xenwatch
893 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 nme-sq
923 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 ipw6_addrconf
928 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kstmp
1106 root 20 0 0 0 0 I 0.0 0.0 0:00.05 kworker/0:4
1587 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 ata_sff

```

6. Regresa a la consola de **CloudWatch** donde dejaste tu lista de alarmas.
7. Haz clic en el nombre de tu alarma **LabCPUUtilizationAlarm**.
8. Monitoriza el gráfico haciendo clic en el botón de actualizar (Refresh) cada 1 minuto. Observarás cómo la línea azul de uso cruza violentamente la línea roja del umbral.
9. El estado de la alarma cambiará de OK a **In alarm** (En alarma).



10. Revisa tu bandeja de entrada del correo electrónico. Deberías tener un mensaje de "ALARM: LabCPUUtilizationAlarm in [Región]" detallando el evento de seguridad. Inserta tu captura aquí: (Captura de pantalla de la consola de CloudWatch mostrando el estado de la alarma en rojo "In alarm" y el gráfico con el pico de CPU superando el 60%)

ALARM: "LabCPUUtilizationAlarm" in US West (Oregon)

Recibidos x

AWS Notifications

para mi

15:51 (hace 1 minuto)

You are receiving this email because your Amazon CloudWatch Alarm "LabCPUUtilizationAlarm" in the US West (Oregon) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [99.74705033528562 (20/03/26 18:46:00)] was greater than the threshold (60.0) (minimum 1 datapoint for OK -> ALARM transition)."

View this alarm in the AWS Management Console:
<https://us-west-2.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-west-2#alarmsV2:alarm/LabCPUUtilizationAlarm>

Alarm Details:

- Name: LabCPUUtilizationAlarm
- Description: CloudWatch alarm for Stress Test EC2 instance CPUUtilization
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [99.74705033528562 (20/03/26 18:46:00)] was greater than the threshold (60.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Friday 20 March, 2026 18:51:40 UTC
- AWS Account: 881805878966
- Alarm Arn: arn:aws:cloudwatch:us-west-2:881805878966:alarm:LabCPUUtilizationAlarm

Threshold:

- The alarm is in the ALARM state when the metric is GreaterThanThreshold 60.0 for at least 1 of the last 1 period(s) of 300 seconds.

Monitored Metric:

- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-0704d9e82a03d8909]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing

State Change Actions:

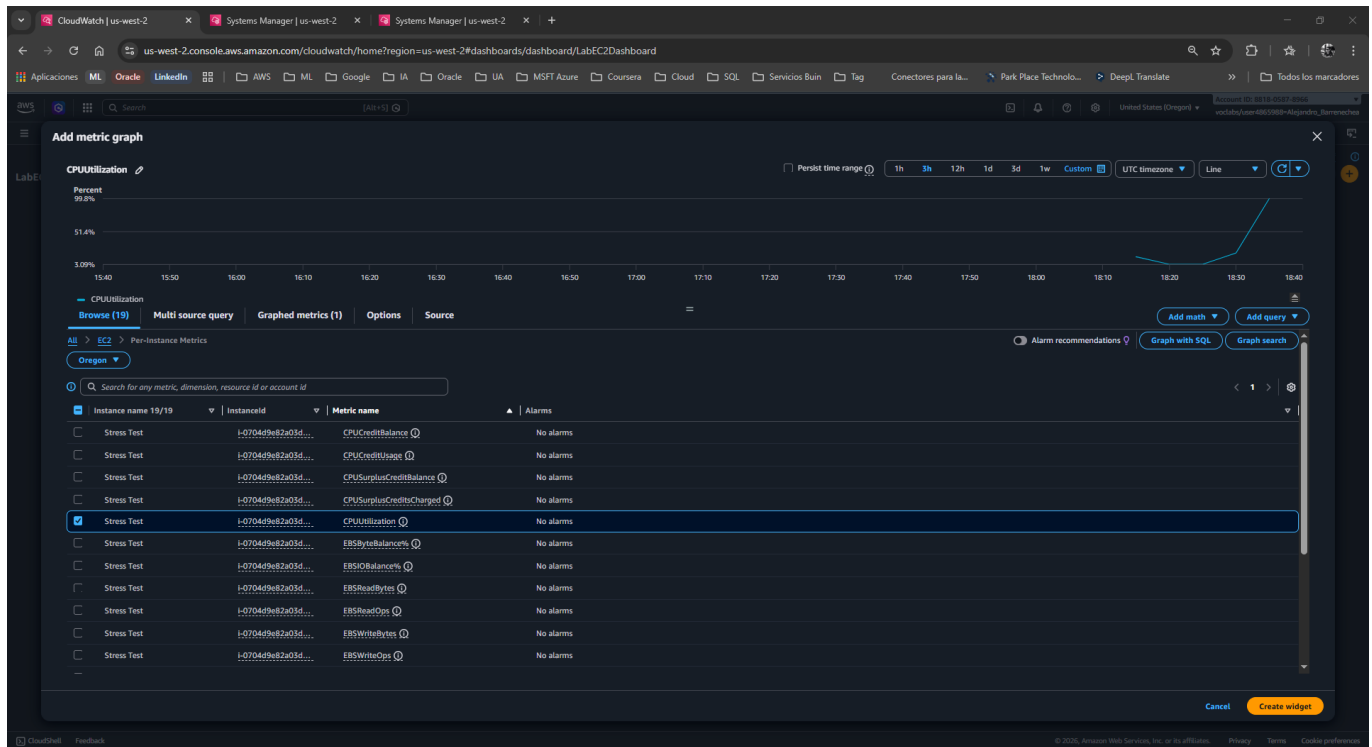
- OK:
- ALARM: [arn:aws:sns:us-west-2:881805878966:MyCwAlarm]
- INSUFFICIENT_DATA:

Tarea 4: Crear un Panel de Control (Dashboard)

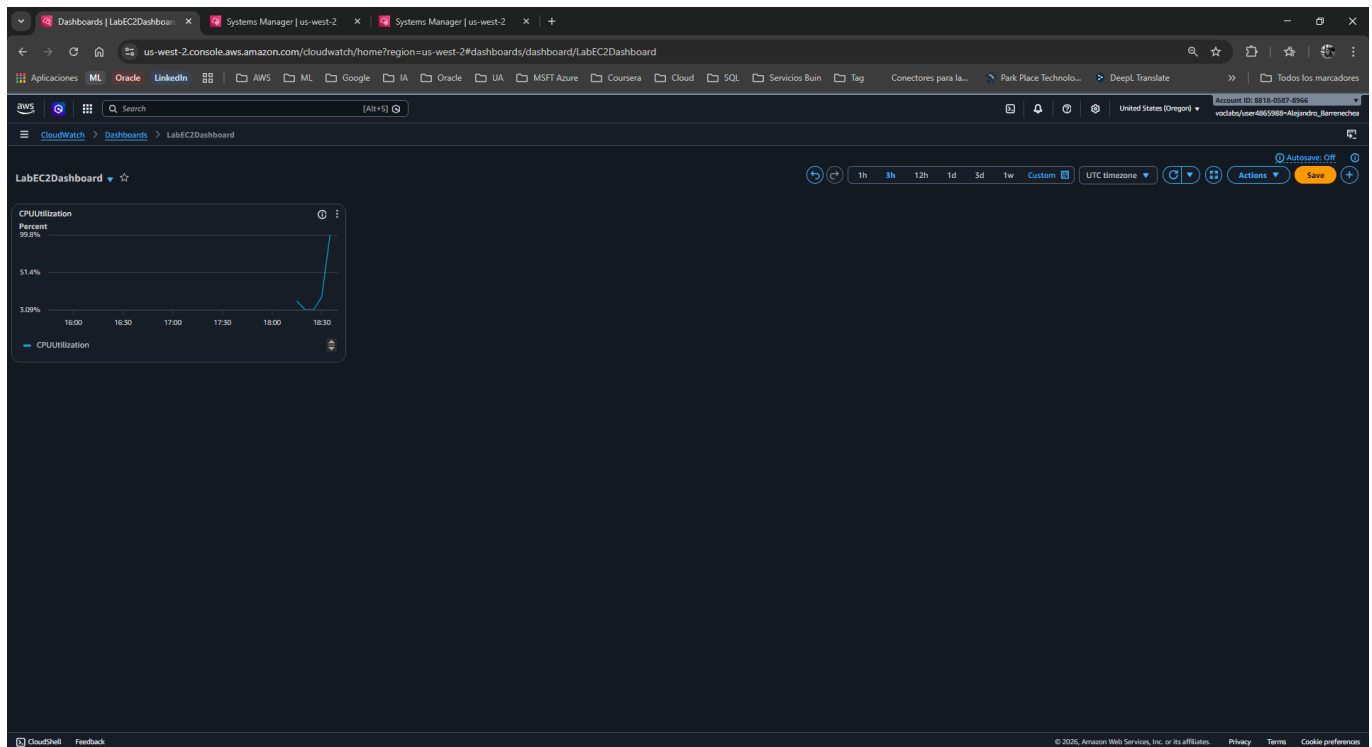
Centralizarás esta visualización para tu centro de operaciones (NOC/SOC).

1. En el panel izquierdo de **CloudWatch**, selecciona **Dashboards**.
2. Haz clic en **Create dashboard**.
3. En **Dashboard name**, escribe **LabEC2Dashboard** y haz clic en **Create dashboard**.
4. En la selección de **Widgets**, elige **Line** (Gráfico de líneas).
5. Selecciona **Metrics**.

6.  Navega a **EC2 > Per-Instance Metrics**.
7. ☒ Marca la casilla de la métrica **CPUtilization** para la instancia **Stress Test**.
8.  Haz clic en el botón naranja **Create widget** (Crear widget).



9.  En la parte superior del Dashboard, asegúrate de hacer clic en **Save dashboard** (Guardar panel). Ahora tienes un centro de monitoreo visual en tiempo real.



Respuestas Analíticas al Caso

¿Por qué es fundamental configurar alarmas sobre el pico de CPU y qué amenaza real simula el comando **stress**?

- **Respuesta:** En ciberseguridad, la disponibilidad es uno de los tres pilares fundamentales (Tríada CIA). Un pico de CPU del 100% sostenido causará Denegación de Servicio (DoS), haciendo que las aplicaciones legítimas de la instancia dejen de responder. El comando `stress` simula un ataque de *Cryptojacking* (donde un atacante inyecta *malware* para minar criptomonedas a costa de tu hardware) o un proceso de un troyano ejecutándose en segundo plano.

¿Qué rol arquitectónico juega Amazon SNS en esta solución y cómo se podría mejorar?

- **Respuesta:** Amazon SNS actúa como el sistema nervioso del monitoreo. Desacopla la detección (CloudWatch) de la acción. En este laboratorio lo usamos para una comunicación de Aplicación a Persona (A2P) enviando un email. Sin embargo, en un entorno de producción avanzado, SNS podría activar una comunicación de Aplicación a Aplicación (A2A), disparando una función de **AWS Lambda** que aisle automáticamente la instancia infectada cambiándole el Security Group, logrando una remediación 100% automatizada.
-